

Building a Secure Malware Lab & Dissecting WannaCry: From ISO to IOCs

A step-by-step guide to setting up an isolated VirtualBox environment for safe ransomware analysis.

The first rule of malware analysis is to create a safe battlefield. This guide walks you through the process of building a custom Home Lab from scratch and deploying a live WannaCry sample inside it. We won't just watch it burn; we will dissect its code, extract Indicators of Compromise and ultimately write our own YARA rules based on the analysis.

CHAPTER 1: LAB SETUP & ISOLATION

Step 1: ISO Selection and VM Configuration

First, we need to acquire a Windows ISO file. I opted for Windows 10 for this lab, though Windows 11 is also an option.

In Oracle VirtualBox, I created a new Virtual Machine and allocated the following resources to ensure smooth performance during analysis:

- 2 Cores
- 4096 MB (4GB)
- 60 GB

Step 2: Post-Installation Adjustments

After the installation, you will notice the screen resolution is fixed at a very small size. To resolve this, we need to install Guest Additions.

1. Click on Devices in the VM menu, then select 'Insert Guest Additions CD image'.
2. Navigate to This PC (File Explorer) and open the CD drive.
3. Run the VBoxWindowsAdditions_amd64installer.
4. Once installed, reboot the system.

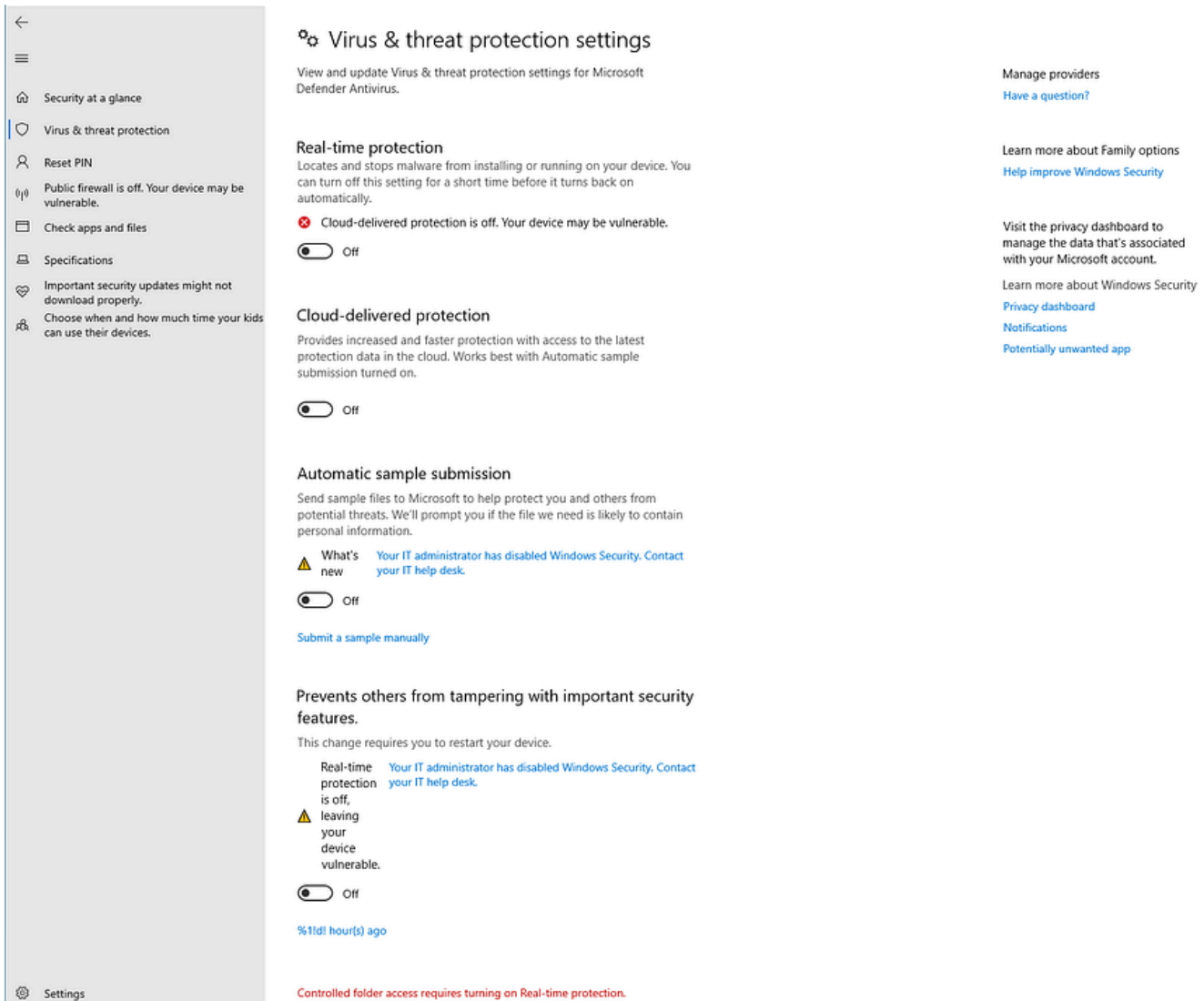
Now, the screen resolution is dynamic, and you can switch to full-screen mode.

Step 3: Disabling Windows Defender

CRITICAL WARNING: Perform the following steps ONLY inside your Virtual Machine. NEVER disable these security features on your main computer, as it will leave you vulnerable to real threats.

To prevent Windows from flagging and deleting our analysis tools (like FlareVM), we must disable the built-in security features:

1. Open the Start Menu, search for “Windows Security” and open the application.
2. Navigate to the tab on the left sidebar.
3. Click on Manage settings (under the Virus & threat protection settings).
4. Toggle OFF all the switches.



1.1

Since the Group Policy Editor is often unavailable in standard Windows 10 Home installations, I achieved the same result by manually modifying the Windows Registry. This step ensures that Windows Defender does not reactivate itself after a restart.

- Press Windows Key+R and type regedit and hit Enter.
- Navigate to the following path:
'HKEY_LOCAL_MACHINE\SOFTWARE\Policies\Microsoft\Windows Defender'
- Right-click on the empty space in the right pane and select New > DWORD (32-bit) Value.
- Name the new key: DisableAntiSpyware (: The naming must be exact for Windows to recognize the command.)
- Double-click the newly created key and change the Value data from 0 to 1.
- Restart the Virtual Machine to apply the changes effectively.

After the system reboots, navigate back to Windows Security > Virus & threat protection to verify the outcome.

Virus & threat protection

Protection for your device against threats.

Your Virus & threat protection is managed by your organization.

 No active antivirus provider. Your device is vulnerable.

1.2

If you see a red warning message stating “**Your Virus & threat protection is managed by your organization**” then congratulations! You have successfully neutralized the defensive mechanisms and prepared the environment for the next phase.

Step 4: Installing FlareVM

Before initiating the automated installation, I highly recommend taking a snapshot of your virtual machine. This acts as a safety net; if anything goes wrong during the installation, you can revert to this clean state instantly.

Preparing PowerShell: By default, Windows restricts script execution. We need to lift this restriction:

1. Open PowerShell as Administrator.
2. Execute the following command: ‘Set-ExecutionPolicy Unrestricted’
3. When prompted, type A (Yes to All) and press Enter.

PRE INSTALL CHECKS:

1.3

Downloading and Launching the Installer:

1. Open the browser inside your VM and navigate to the official Mandiant Flare-VM GitHub repository (github.com/mandiant/flare-vm).

2. Download the repository as a ZIP file and extract it to your Desktop.
 3. Return to your PowerShell window and navigate to the extracted folder using the `cd` command
 4. Run the installer script by typing: `.\installer.ps1`
 5. If prompted with a security warning, type `R` (Run once) to authorize the script.
 6. A setup window will appear. Do not change any settings. Simply keep the default paths, click Continue, select the default packages, and click Install.
 7. The installation will take a couple hours and the VM will restart automatically multiple times. Just let it run until you see the new FlareVM desktop.
-

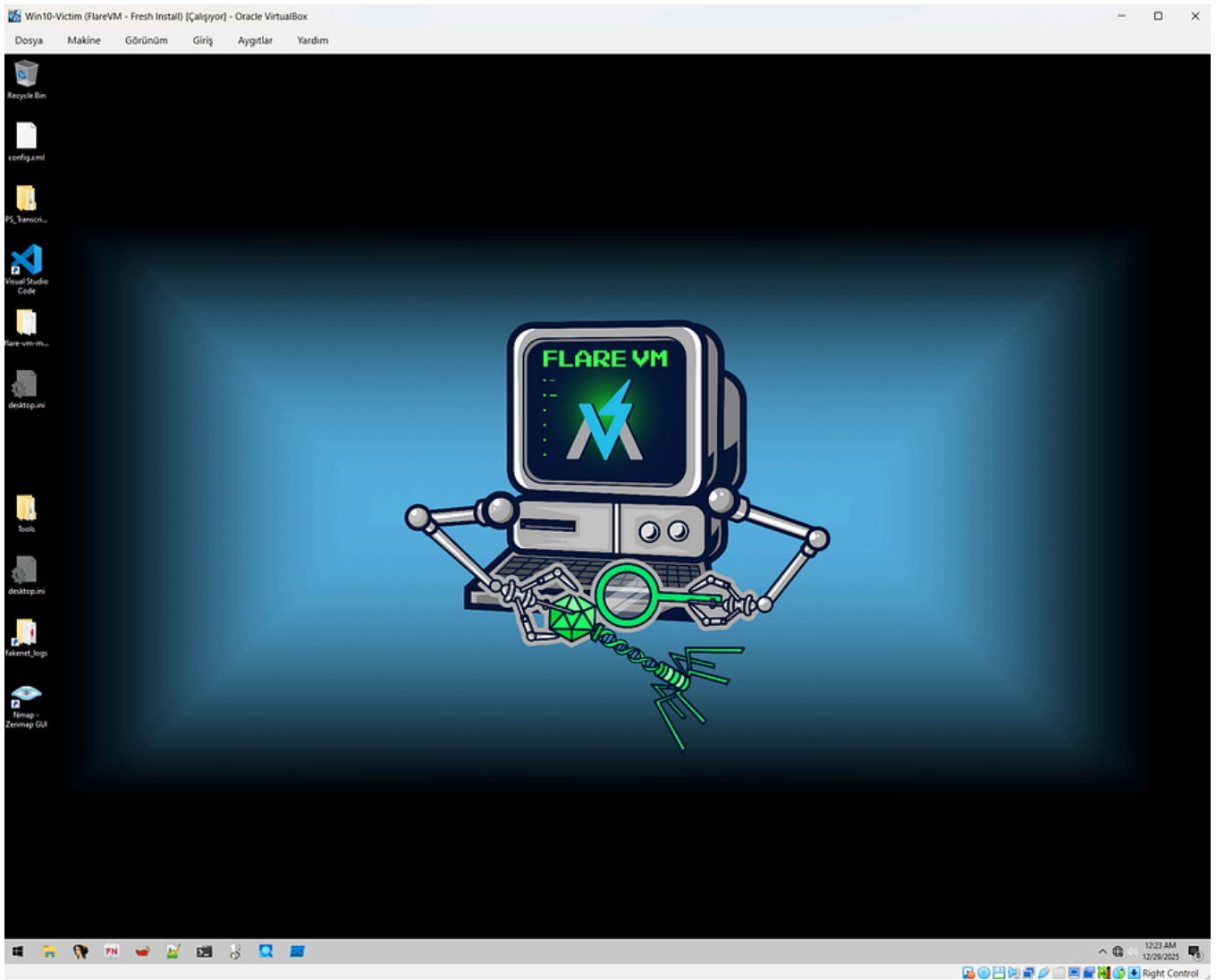
STEP 5: Network Configuration & Isolation

Once the installation is complete, you will notice the wallpaper features a red flare design with a warning indicating that the internet is enabled. To create a safe environment we must sever this connection.

Navigate to **Devices** → **Network** → **Network Settings**. Change the adapter mode from 'NAT' to 'Host-only Adapter' and save. Next, ensure total isolation by disabling Shared Folders, Shared Clipboard, and Drag'n'Drop features under the Devices menu. We don't want any bridge left between the VM and the Host.

After successfully cutting the connection, the wallpaper should automatically update (turning blue). To be absolutely sure, open the terminal and try to **ping google.com**. The request should fail.

Pro Tip: With the configurations complete, take another snapshot immediately. This will be your "Clean State." Trust me, nobody wants to wait 4 hours for a re-installation if something goes wrong.



1.4

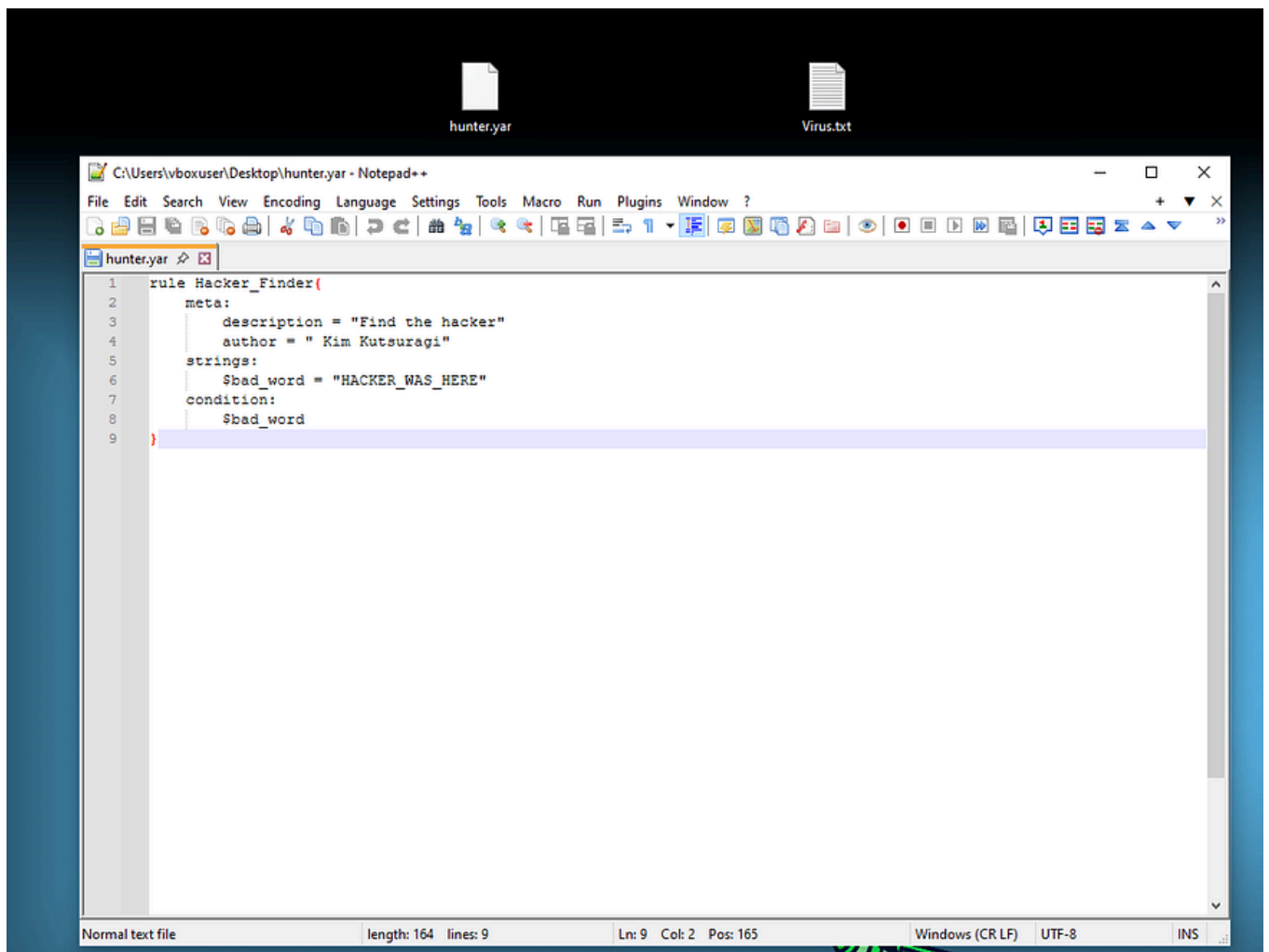
CHAPTER 2: Tool Verification (Sanity Check)

Before diving deep into the WannaCry binary, we need to ensure our analysis environment is functioning correctly. We will start by learning the basics of YARA (Yet Another Ridiculous Acronym), the industry standard tool for pattern matching and malware identification.

In this step, we will create a benign “dummy” virus and attempt to detect it with a custom YARA rule.

1. Create a new text file on your desktop named virus.txt
2. Open it and write the following string inside: “”This string will serve as our malicious signature.
3. Create a new file named hunter.yar.
4. Open it with a text editor (like Notepad++) and paste the following code from picture 1.5.
5. Open your terminal (CMD or PowerShell).
6. Navigate to the desktop (or the folder where you created the files).
7. Run the following command in picture 1.6.

Note: The dot (.) at the end of the command tells YARA to scan all files in the current directory.



1.5

```
FLARE-VM Mon 12/29/2025 19:39:09.61  
C:\Users\vboxuser\Desktop>yara64 hunter.yar .  
Hacker_Finder .\hunter.yar  
Hacker_Finder .\Virus.txt  
  
FLARE-VM Mon 12/29/2025 19:39:14.19  
C:\Users\vboxuser\Desktop>
```

1.6

If everything is working correctly, YARA should output the name of our rule and the file it matched like in picture 1.6. You might have noticed something interesting in the results. YARA likely flagged two files instead of one.

Why?

Since we instructed YARA to scan the current directory (.), it scanned every single file, including our rule file. Because hunter.yar contains the string definition "HACKER_WAS_HERE", so it technically matches its own rule! This is a great example of a false positive.

CHAPTER 3: SECURE MALWARE TRANSFER

CRITICAL WARNING:

DO NOT open, extract, or run the downloaded files on your Host Machine. DO NOT forget to disable Shared Folders immediately after the transfer. DO NOT leave any malicious residue on your main system

In this chapter, we will source a live sample of WannaCry. We have two options: downloading it directly inside the VM or downloading it to the Host and transferring it. I prefer the Host-to-VM transfer method.

First you need to turn off the defender for a short period of time.

Gentle Reminder Warning: (Dropper vs. Payload)

Before we download anything, let me share a critical lesson I learned the hard way during this analysis. Not all “WannaCry” samples are created equal.

You will encounter two main hash values in the wild. Knowing the difference will save you hours of debugging:

- 24d004a104d4d54034dbcffc2a4b19a11f39008a575aa614ea04703480b1022c
- This is the worm component. It contains the Kill Switch logic we want to analyze. Download this one.
- ed01ebfbc9eb5bbea545af4d01bf5f1071661840480439c6e5babe8e080e41aa
- This is just the encryption component extracted from the dropper. It does not check for the kill switch. If you analyze this, you will miss the network indicators.

In this walkthrough, I proceeded with the Payload sample. As a result, in the upcoming chapters I was unable to trigger the Kill Switch mechanism because this specific file doesn't have it.

If you follow my steps exactly: You will be analyzing the Payload.

My Recommendation: If your goal is to catch the Kill Switch, ignore my file choice and download the Dropper instead.

Step 1: Preparing the “Containment Zone” on Host

Windows Defender will instantly eat these files if we aren't careful. Instead of turning off our entire antivirus (which is risky), we will create a specific Exclusion Zone.

1. Create a folder on your desktop named Careful_Malicious.
2. Go to Windows Security → Virus & threat protection.
3. Click Manage settings → Scroll down to Exclusions → Add or remove exclusions.
4. Select Add an exclusion → Folder → Choose your folder.

Now, anything inside this folder is invisible to Windows Defender.

Step 2: Sourcing the Sample

We will use the famous malware repository, TheZoo.

1. Navigate to the [ytisf/theZoo](https://github.com/ytisf/theZoo) GitHub repository.
2. Go to theZoo/malware/Binaries/Ransomware.WannaCry.
3. Download the ZIP file directly into your Careful_Malicious folder.
4. Do not unzip it yet.

Step 3: The Transfer

Now we move the package into the isolated FlareVM.

1. Open VirtualBox/VMware settings for your FlareVM.
2. Go to Shared Folders.
3. Add your Careful_Malicious folder as a shared folder (Auto-mount enabled).
4. Boot up the FlareVM.
5. Inside the VM, navigate to the Shared Drive (Z:).
6. Copy the ZIP file from the share to the VM's Desktop.

Step 4: Burning the Bridge (Crucial!)

Once the file is safely inside the VM, we must cut the connection immediately to prevent the worm from crawling back to the host via the shared folder vulnerability (SMB).

1. Go back to VirtualBox/VMware settings.
2. Remove/Delete the Shared Folder configuration.
3. On your Host Machine, delete the Careful_Malicious folder and the ZIP file inside it.
4. Remove the Exclusion from Windows Defender.

We are now safe. The malware is trapped inside the VM, and the bridge to the outside world has been destroyed. Let the analysis begin.

CHAPTER 4: Basic Static Analysis

Now that the sample is safely inside our isolated FlareVM environment, it's time to get our hands dirty. But before we execute anything we must perform Static Analysis. This means studying the malware without running it.

Step 1: Extraction & Password

Locate the file Ransomware.Wannacry.zip we transferred to the desktop.

1. Right-click and Extract Here.
2. If it asks for a password, the password is 'infected'

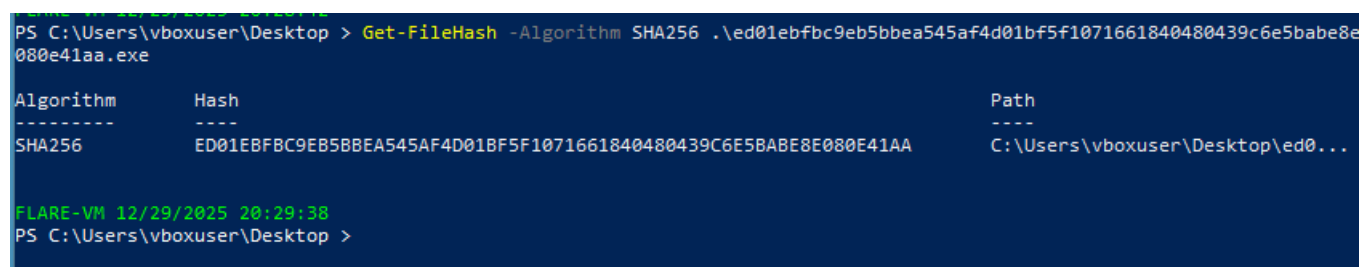
Once extracted, you will see an executable file named ed01...exe

Note: As much as I understand, in malware repositories files are often renamed to their SHA256 hash values to prevent accidental execution and for easy identification.

Step 2: Hash Verification

Even though the filename looks like a hash, we never trust surface level data. We must verify the file's integrity ourselves. We will use PowerShell to calculate the file's hash and compare it with known threat intelligence.

1. Open PowerShell
2. Navigate to the Desktop (or wherever you extracted the file).
3. Run the following command from picture 1.7



```
PS C:\Users\vboxuser\Desktop > Get-FileHash -Algorithm SHA256 .\ed01ebfbc9eb5bbea545af4d01bf5f1071661840480439c6e5babe8e080e41aa.exe

Algorithm      Hash
-----
SHA256         ED01EBFBC9EB5BBEA545AF4D01BF5F1071661840480439C6E5BABE8E080E41AA
Path
-----
C:\Users\vboxuser\Desktop\ed0...

FLARE-VM 12/29/2025 20:29:38
PS C:\Users\vboxuser\Desktop >
```

1.7

Step 3: Analyzing the Result

The command will return the same hash value of the file name.

Confirmation: The calculated hash matches the filename exactly. Referring back to our “Gentle Reminder” note in Chapter 3, we can now confirm 100% that we are holding the Payload, not the Dropper.

Step 4: Threat Intelligence Verification (VirusTotal)

Calculating the hash is useless if we don't compare it against a database. We will use VirusTotal, the industry standard search engine for malware to cross reference our findings.

1. Copy the SHA256 string we generated in the previous step.
ED01EBFBC9EB5BBEA545AF4D01BF5F1071661840480439C6E5BABE8E080E41AA
2. Open your browser on your host machine and navigate to [VirusTotal.com](https://www.virustotal.com) and paste the hash value.

The screen should light up red like in picture 1.8 This confirms one thing and that is that the file is indeed Malicious.

68

/ 72

Community Score

-2976

68/72 security vendors flagged this file as malicious

Reanalyze

Similar

More

ed01ebfbc9eb5bbea545af4d01b5f1071661840480439c6e5babe8e080e41aa

Size

3.35 MB

Last Analysis Date

5 hours ago

EXE

diskpart.exe

peexe

checks-cpu-name

runtime-modules

detect-debug-environment

checks-user-input

checks-disk-space

direct-cpu-clock-access

overlay

via-tor

executes-dropped-file

checks-bios

long-sleeps

checks-network-adapters

malware

self-delete

calls-wmi

ssh-communication

macro-create-ole

DETECTION

DETAILS

RELATIONS

BEHAVIOR

COMMUNITY 30+

Join our Community and enjoy additional community insights and crowdsourced detections, plus an API key to automate checks.

Popular threat label

ransomware.wannacry/wanna

Threat categories

ransomware

trojan

Family labels

wannacry

wanna

wannacryptor

Security vendors' analysis

Do you want to automate checks?

AhnLab-V3	Trojan.Win32.WannaCryptor.R200571	Alibaba	Ransom:Win32/WannaCry.ali1020010
AliCloud	RansomWare	ALYac	Trojan.Ransom.WannaCryptor
Antiy-AVL	Trojan[Ransom]/Win32.Wanna	Arcabit	Trojan.Ransomware.Y
Arctic Wolf	Unsafe	Avast	Win32:WanaCry-A [Trj]
AVG	Win32:WanaCry-A [Trj]	Avira (no cloud)	TR/Ransom.JB
Baidu	Win32.Trojan.WannaCry.c	BitDefender	Trojan.Ransomware.Y
Bkav Pro	W32.WanaCrypt.BTTC.Worm	ClamAV	Win.Ransomware.Wannacryptor-994018...
CrowdStrike Falcon	Win/malicious_confidence_100% (W)	CTX	Exe.ransomware.wannacry
Cynet	Malicious (score: 100)	DeepInstinct	MALICIOUS
DrWeb	Trojan.Encoder.11432	Elastic	Malicious (high Confidence)
Emsisoft	Trojan.Ransomware.Y (B)	eScan	Trojan.Ransomware.Y
ESET-NOD32	Win32/Filecoder.WannaCryptor.D Trojan	Fortinet	W32/WannaCryptor.6F871tr.ransom
GData	Win32.Trojan-Ransom.WannaCry.A	Gridinsoft (no cloud)	Malware.Win32.Gen.botlsc54409
Huorong	Ransom/Wannacry.j	Ikarus	Trojan-Ransom.WannaCry
Jiangmin	Trojan.Wanna.eo	K7AntiVirus	Trojan (0050d7171)
KTGW	Trojan (0050d7171)	Kaspersky	Trojan-Ransom.Win32.Wanna.zbu
Kingsoft	Win32.Troj.Undef.a	Lionic	Trojan.Win32.Wanna.toNn
Malwarebytes	Generic.Malware.Gen.DDS	MaxSecure	Trojan.Ransom.Wanna.d
McAfee Scanner	THED01EBFBC9EB	Microsoft	Ransom:Win32/WannaCrypt!pz
NANO-Antivirus	Trojan.Win32.Ransom.eoptnj	Palo Alto Networks	Generic.ml
Panda	Trj/RansomCrypt.K	QuickHeal	Ransom.WannaCrypt.A4
Rising	Ransom.WanaCrypt!1.AAD9 (CLASSIC)	Sangfor Engine Zero	Ransom.Win32.Save.WannaCry
SecureAge	Malicious	SentinelOne (Static ML)	Static AI - Suspicious PE

WannaCry is famous for a specific “feature” hardcoded into its binary: The Kill Switch. Here is the logic:

The Goal: We want to find this URL inside the file strings.

(Spoiler from the future: I couldn't find it because I downloaded the payload instead of the dropper)

The Hunt: Using Strings & FLOSS

We will attempt to extract readable text strings from the binary to find any HTTP links.

Attempt 1: Standard Strings

Open PowerShell and run:

```
strings .\ed01ebfbc9eb5bbea545af4d01bf5f1071661840480439c6e5babe8e080e41aa.exe |  
findstr "http"
```

Attempt 2: Advanced Extraction (FLOSS)

When standard strings failed to return the specific URL. I escalated to FLOSS which is better at extracting hidden strings.

```
(floss .\ed01ebfbc9eb5bbea545af4d01bf5f1071661840480439c6e5babe8e080e41aa.exe |  
findstr "http")
```

Both commands returned no results regarding the kill switch domain because we are analyzing the payload(At least I am.)

A Curious Discovery: "Global Customer Service"

While manually scrolling through the strings output to find the missing URL, I stumbled upon something else. I noticed a long list of folder names corresponding to different languages (Bulgarian, Chinese, English, Turkish, etc.).

It seems the malware authors put significant effort into User Experience (UX). They ensured the ransom note is translated into almost every major language, essentially creating a "Global Customer Service" for their victims. You have to admire the scale of operation.

Note: Since this was my first encounter with WannaCry, this was a surprise to me!

```

IAGG
msg/m_bulgarian.wnry
"t=)
msg/m_chinese (simplified).wnry
"t=.|Vbq-
msg/m_chinese (traditional).wnry
"t=.
msg/m_croatian.wnry
msg/m_czech.wnry
msg/m_danish.wnry
msg/m_dutch.wnry
msg/m_english.wnry
"t=m
msg/m_filipino.wnry
"t=?
msg/m_finnish.wnry
"t=-
msg/m_french.wnry
msg/m_german.wnry
&XZR%
msg/m_greek.wnry
"t=x
msg/m_indonesian.wnry
"t=j%
msg/m_italian.wnry
"t=x-
msg/m_japanese.wnry
msg/m_korean.wnry
msg/m_latvian.wnry
msg/m_norwegian.wnry
msg/m_polish.wnry
msg/m_portuguese.wnry
"t=@W
msg/m_romanian.wnry
msg/m_russian.wnry
"t=3M
msg/m_slovak.wnry
msg/m_spanish.wnry
msg/m_swedish.wnry
msg/m_turkish.wnry
msg/m_vietnamese.wnry
r.wnry
Jcg4k_
s.wnry
t.wnry
*($:{
taskdl.exe
taskse.exe

```

1.9

Attempt 3: HxD

Not satisfied with the command-line results, I decided to go deeper. I opened HxD (Hex Editor) and dragged the malicious file into it to inspect the raw binary code manually.

I performed a text search for generic terms like http, www, and .com.

Result: Still nothing. This final manual check cemented my conclusion: The Kill Switch URL is simply not present in this file.

Technical Hurdle: PeStudio Crash

I attempted to use PeStudio for a deeper look at the imports and libraries. However, malware analysis requires resource adaptability.

- PeStudio repeatedly crashed my FlareVM environment.
- Instead of wasting hours troubleshooting the tool, I decided to skip it. In real world incident response, time is money. If a tool fails, you move to the next one.

Conclusion of Static Analysis

We have extracted as much information as possible without executing the file. Here is what we know so far:

1. It is definitely WannaCry (verified via Hash check).
2. It includes “Global Customer Service” language support.
3. It appears to be missing the Kill-Switch.

Next Stop: Dynamic Analysis.

CHAPTER 4: DYNAMIC ANALYSIS

In this chapter, we move from theory to action. My primary goal was to capture the “Kill Switch” URL that eluded us during the static analysis. To do this, I set up a network trap.

Step 1: The Trap (FakeNet)

I launched Fakenet, a tool that simulates the internet.

1. If the malware tries to contact the “Kill Switch” domain, FakeNet will intercept the request, say “Yes, I am here” and theoretically stop the malware from encrypting the system.
2. Capture the HTTP request and see the domain name in the logs.

I ran the malware as Administrator and watched the FakeNet console like a hawk.

1. Still nothing. FakeNet did not capture any HTTP requests (Picture 2.0) of a strange domain. Instead, the malware immediately executed its encryption routine. Within seconds my wallpaper changed and my files were locked.

I was forced to perform an emergency rollback to my clean VM snapshot.

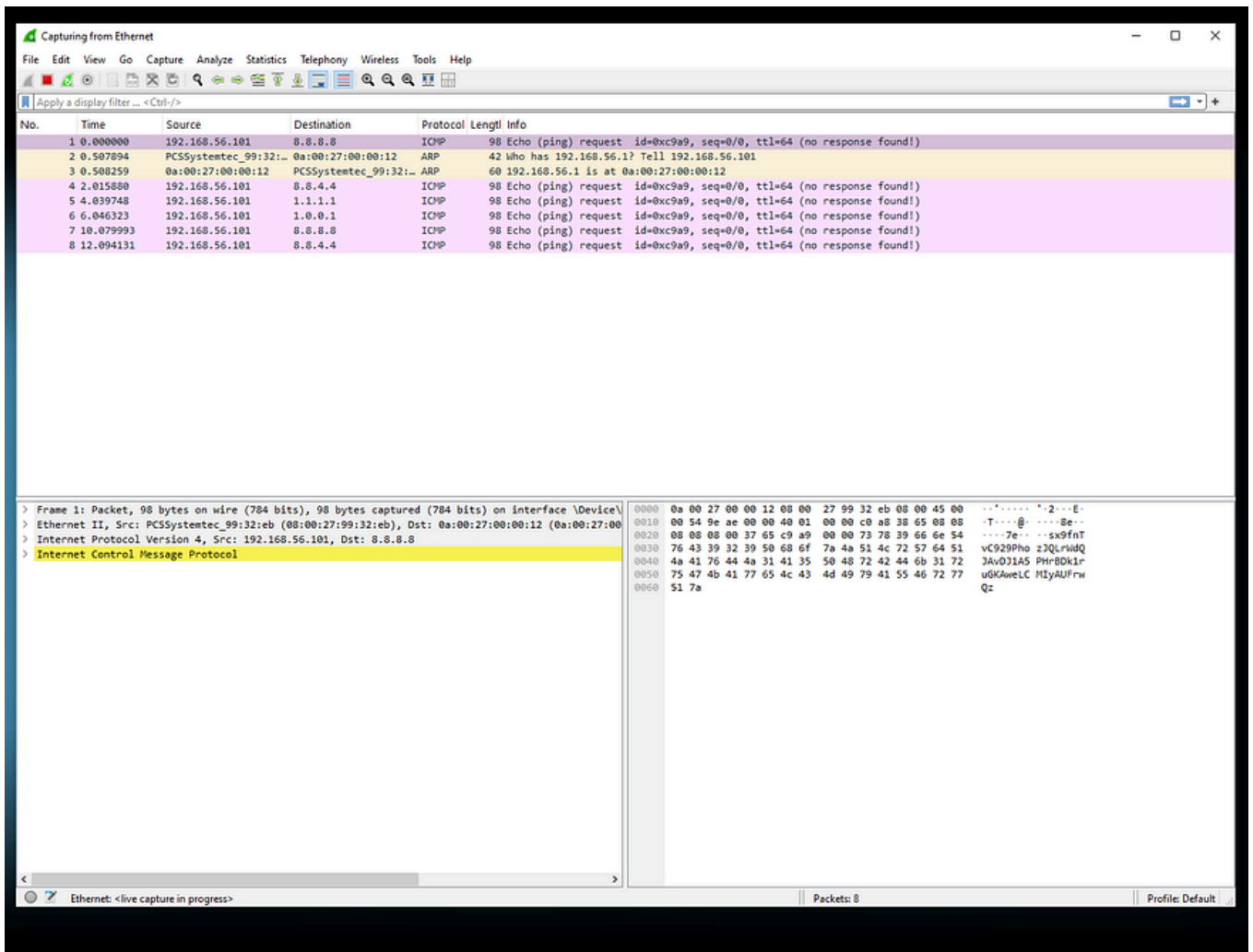
[illegible]

2.0

Step 2: Deep Packet Inspection (Wireshark)

Not convincing myself with the FakeNet results, I decided to “change guns” and bring out the heavy artillery. I let the virus fully infect the VM while listening to the network traffic with Wireshark.

- Capture the specific DNS query (the “Hello” message) sent to the Kill Switch domain.
- Fail. I filtered for dns and http traffic expecting to see a POST request. There was nothing but standard background noise. The problem was consistent.

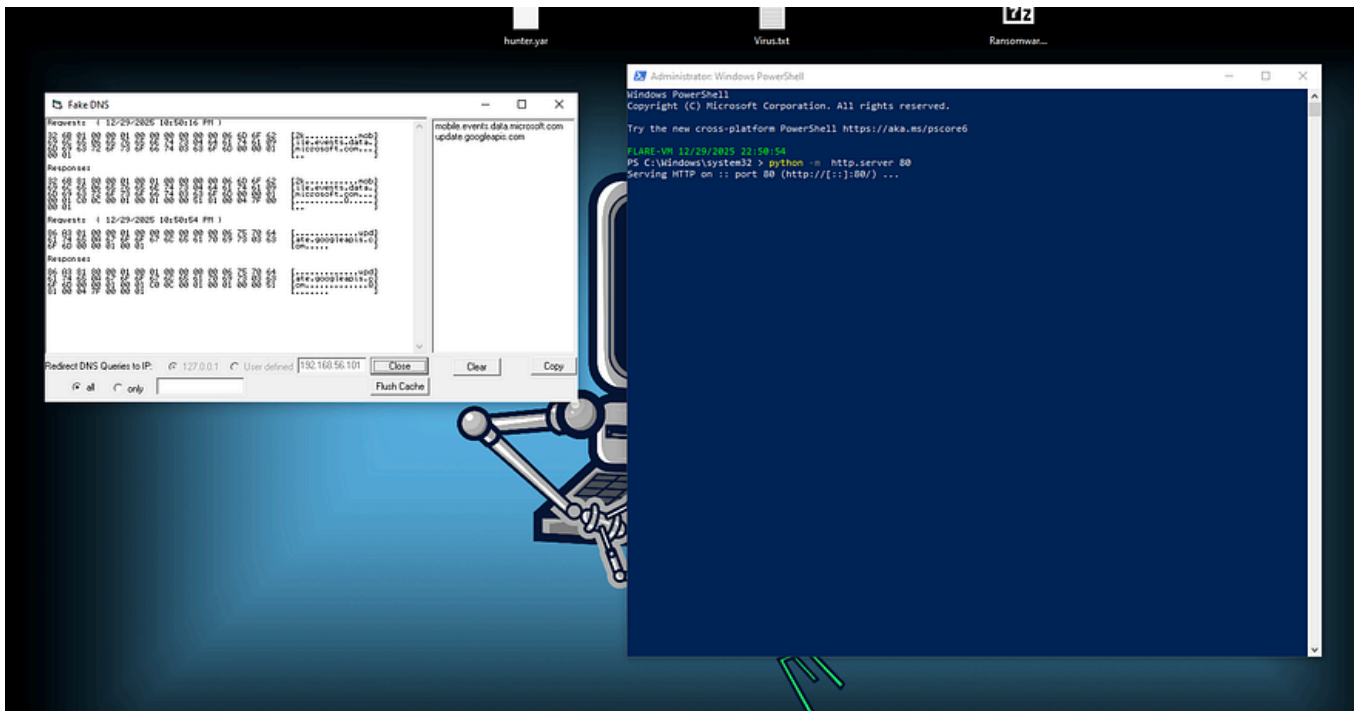


2.1

Step 3: FakeDNS

Thinking maybe Wireshark missed the handshake or FakeNet was too complex so I simplified the approach using FakeDNS.

The console remained empty (Pic 2.2). No DNS requests were made. This was becoming highly suspicious. As much as I read, a standard WannaCry sample should reach out immediately!



2.2

Step 4: Ghidra

At this point, I was frustrated. I decided to stop guessing and look at the code structure itself. I opened the file in Ghidra, the NSA's own reverse engineering tool.

Ghidra is a complex beast to handle. Staring at thousands of lines of assembly code was overwhelming for my current skill level. Instead of pretending to be an expert reverse engineer, I used AI tools for guidance.

I researched specific memory addresses and function calls associated with WannaCry's kill switch to know exactly where to look in the decompiled code.

Even with external guidance pointing me to where the URL should be, it was nowhere to be found. The code blocks responsible for the domain check were simply missing from this binary.

@Please_Read_Me@.txt - Notepad

File Edit Format View Help

: What's wrong with my files?

: Oops, your important files are encrypted. It means you will not be able to access them anymore until they are decrypted. If you follow our instructions, we guarantee that you can decrypt all your files quickly and safely! Let's start decrypting!

: What do I do?

: First, you need to pay service fees for the decryption.
Please send \$300 worth of bitcoin to this bitcoin address: 115p7UFPmgoj1pMvkpHjcrdFJNXj6LrLn
Next, please find an application file named "@WanaDecryptor@.exe". It is the decrypt software.
Run and follow the instructions! (You may need to disable your antivirus for a while.)

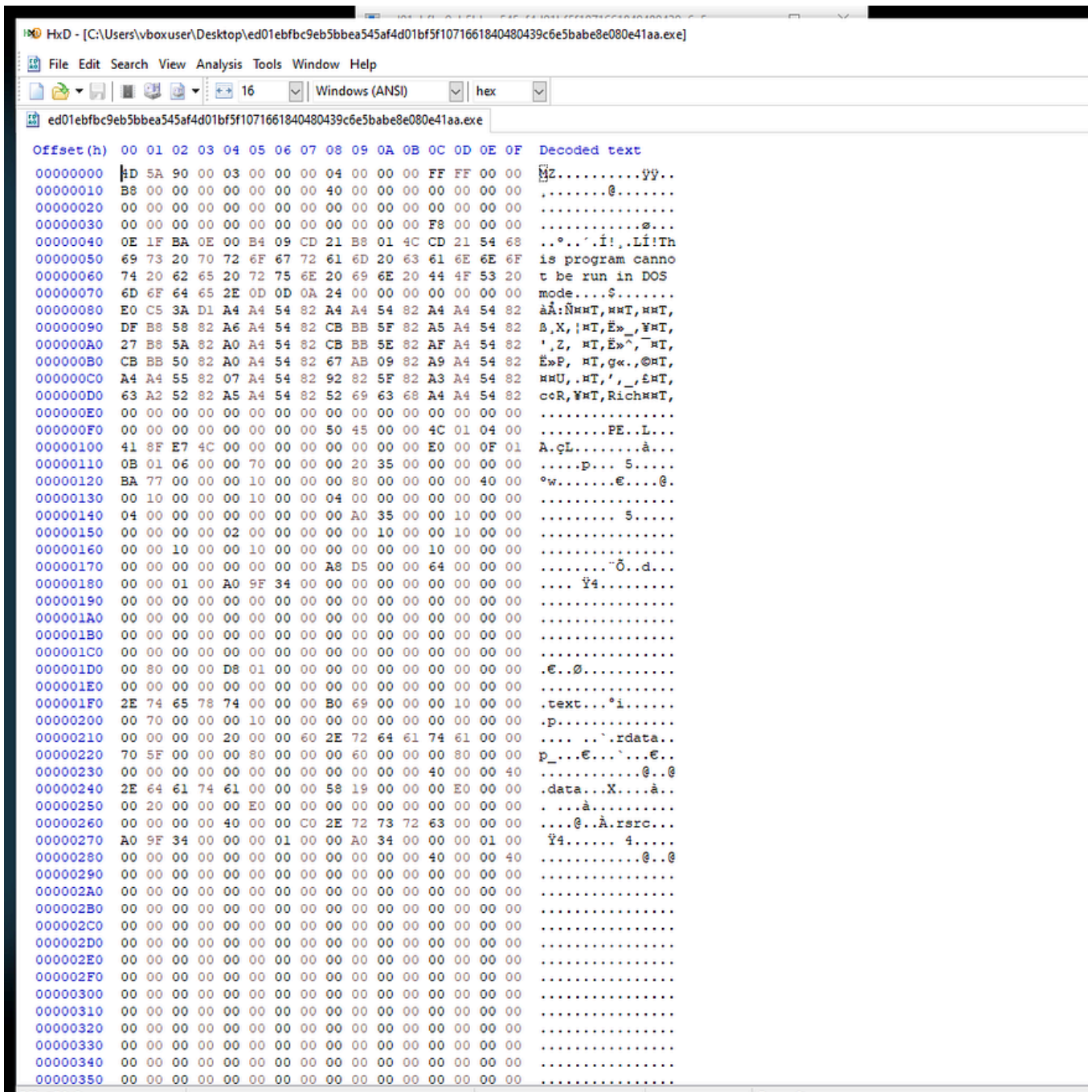
: How can I trust?

: Don't worry about decryption.
We will decrypt your files surely because nobody will trust us if we cheat users.

If you need our assistance, send a message by clicking <Contact Us> on the decryptor window.



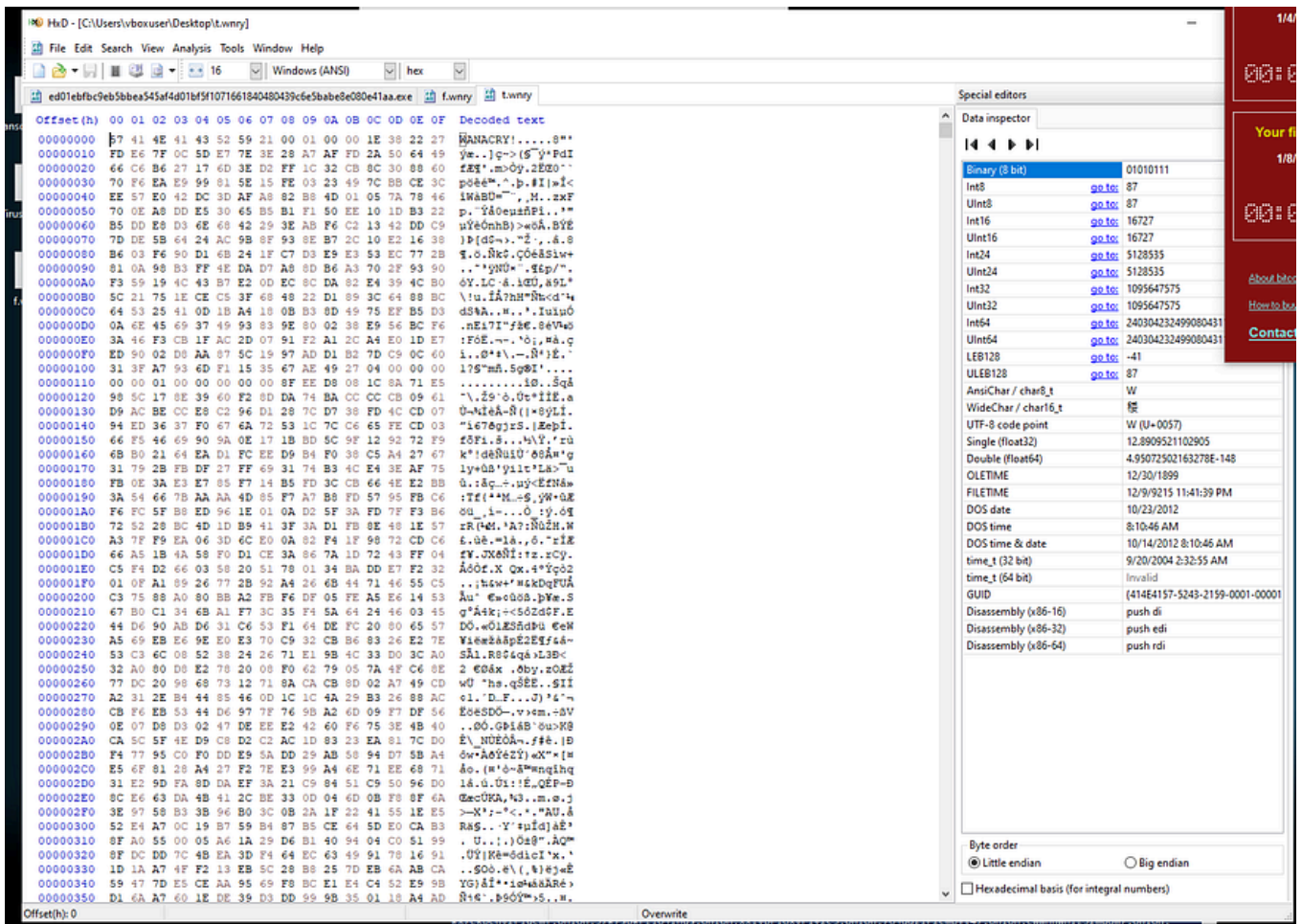
Wannacry?



2.4

Picture 2.4: When we look at the very beginning of the file in our Hex Editor, the first two decoded characters are M and Z.

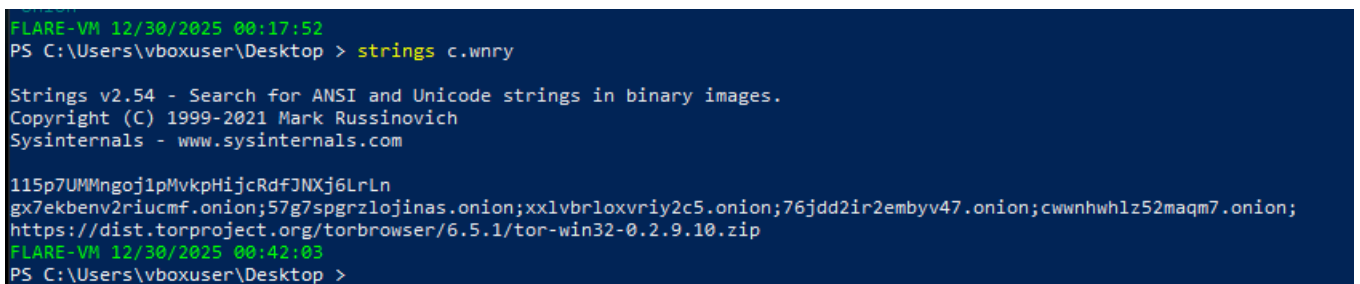
- 5A 4D
- This stands for “Mark Zbikowski” (a Microsoft architect). It is the universal signature for Windows Executable files.
- We will add `uint16(0) == 0x5A4D` to our YARA rule to ensure we are only scanning .exe files.



2.5

Picture 2.5: Next, we inspected the t.wnry file, which is part of the encryption process. Inside the header, the malware explicitly brands itself.

- WANACRY!
- This is the marker the malware stamps on files to know they are already encrypted.
- This is a high fidelity string. If a file contains this text, it is undoubtedly malicious.



2.6

Picture 2.6: Finally our deep dive revealed the malware's communication channels. In the configuration block we uncovered:

- .onion links used to communicate with the hackers via the Dark Web.
- The address where victims are forced to send money.
- These are unique IoC. Adding these to our rule proves we detected the specific variant's network infrastructure.

CHAPTER 5: REPORTING & IOC EXTRACTION

After completing the static and dynamic analysis phases, the final step is to transform our raw findings into actionable intelligence for Blue Teams and decision makers.

A critical takeaway from this analysis was understanding the multi staged architecture of WannaCry. The investigation revealed that the malware is not a single file but operates with distinct components for propagation and encryption.

1. Case Analysis: Dropper vs. Payload Distinction

During the initial phase of the analysis, the “Kill Switch” mechanism was not triggered. A root cause analysis revealed that we were analyzing the dropped payload rather than the initial dropper.

2. Indicators of Compromise (IoC)

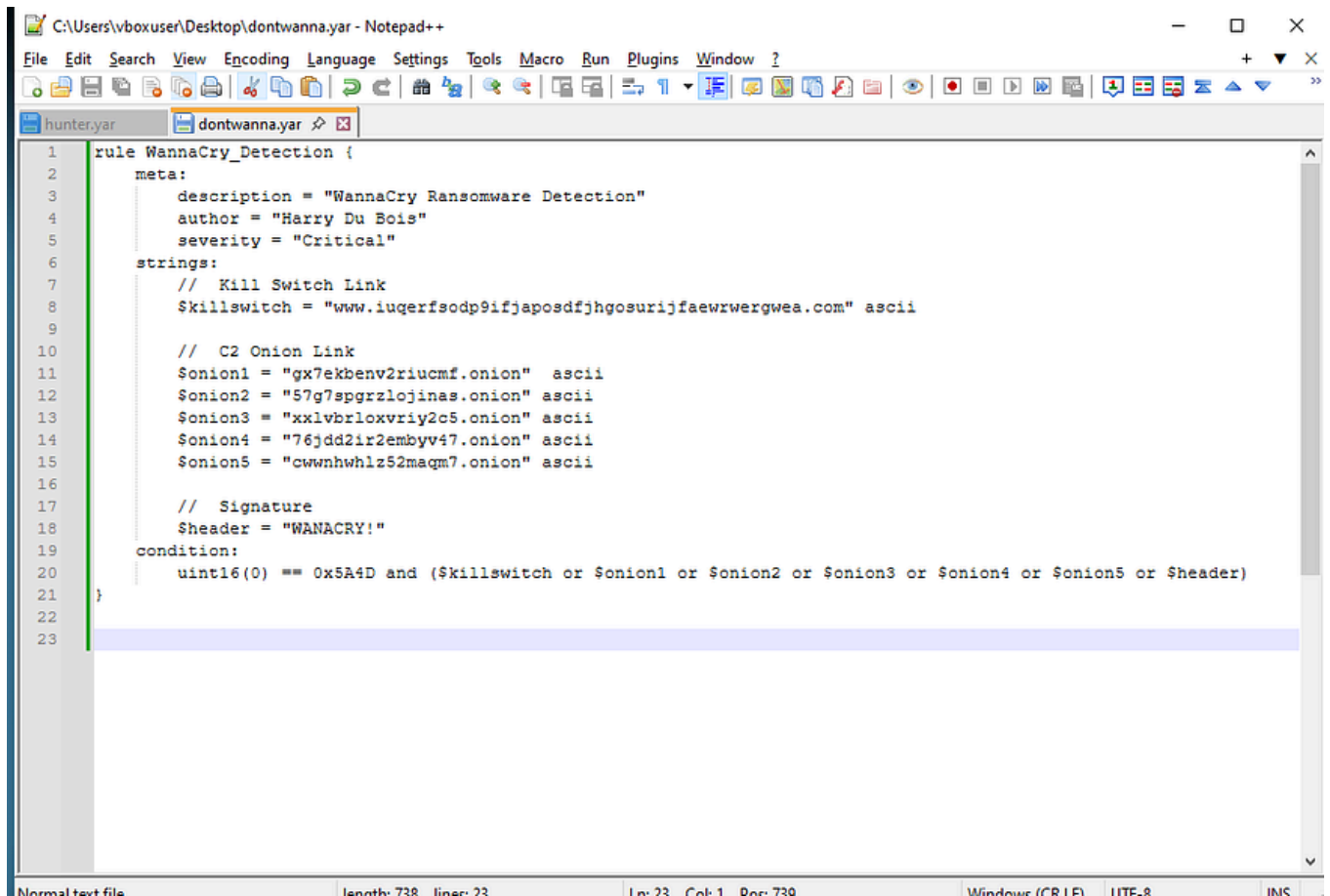
File Indicators (SHA256 Hashes)

1. mssecsvc.exe
24d004a104d4d54034dbcffc2a4b19a11f39008a575aa614ea04703480b1022cThe Worm component. Contains the Kill Switch logic.
2. tasksche.exe*
ed01ebfbc9eb5bbea545af4d01bf5f1071661840480439c6e5babe8e080e41aa The Encrypter component. Encrypts files and appends
3. The GUI executable displayed to the victim.

*The name's replaced with hash value if you download it from thezoo.

3. YARA Rule

The following YARA rule can be used to detect both the dropper and the payload components within the environment. I wrote these rules based on my findings from both static and dynamic analyses.



```
1 rule WannaCry_Detection {
2   meta:
3     description = "WannaCry Ransomware Detection"
4     author = "Harry Du Bois"
5     severity = "Critical"
6   strings:
7     // Kill Switch Link
8     $killswitch = "www.iuqerfsodp9ifjaposdfjhgosurijfaewrgwea.com" ascii
9
10    // C2 Onion Link
11    $onion1 = "gx7ekbenv2riucmf.onion" ascii
12    $onion2 = "57g7spgrzlojinaz.onion" ascii
13    $onion3 = "xxlvbrloxvriy2c5.onion" ascii
14    $onion4 = "76jdd2ir2embyv47.onion" ascii
15    $onion5 = "cwwnhwhlz52maq7.onion" ascii
16
17    // Signature
18    $header = "WANACRY!"
19  condition:
20    uint16(0) == 0x5A4D and ($killswitch or $onion1 or $onion2 or $onion3 or $onion4 or $onion5 or $header)
21 }
22
23
```

2.7

4. Executive Summary & Mitigation

WannaCry is a self-propagating ransomware that exploits the MS17-010 vulnerability in the Windows SMB protocol. My analysis could not confirm the existence of a “Kill Switch” mechanism in this sample. However it is widely known that registering and sinkholing the hardcoded domain prevents the dropper component from encrypting the system.

Recommendations:

- Ensure the Kill Switch domain resolves to a safe IP address (sinkhole) at the DNS level.
- Apply the MS17-010 security update to all Windows systems immediately.
- Block inbound SMB traffic (Port 445) from the external network and segment internal traffic to prevent lateral movement.

Author’s Note:

All technical content and methodologies in this article are my own original work. I have utilized AI tools solely for linguistic refinement and paraphrasing to ensure clarity and professional tone.

Closing Note:

I recently earned my CompTIA Security+ certification and am now actively bridging the gap between theory and practice through Home Labs. I enjoy documenting this learning journey on Medium. There is much more to come, so stay tuned!